

Compose once. Render through modules.

A lean Java document core with a semantic model, deterministic layout geometry and opt-in output backends.



io.github.demchaav:graph-compose-core

+ choose a renderer module

THE MODULE GRAPH



graph-compose-core

semantic graph

authoring · nodes · layout services

NO PDFBOX / POI



render-pdf

fixed layout



render-pptx

fixed layout



render-docx

semantic

● graph-compose-core

lean engine

● graph-compose

drop-in PDF

● ServiceLoader SPI

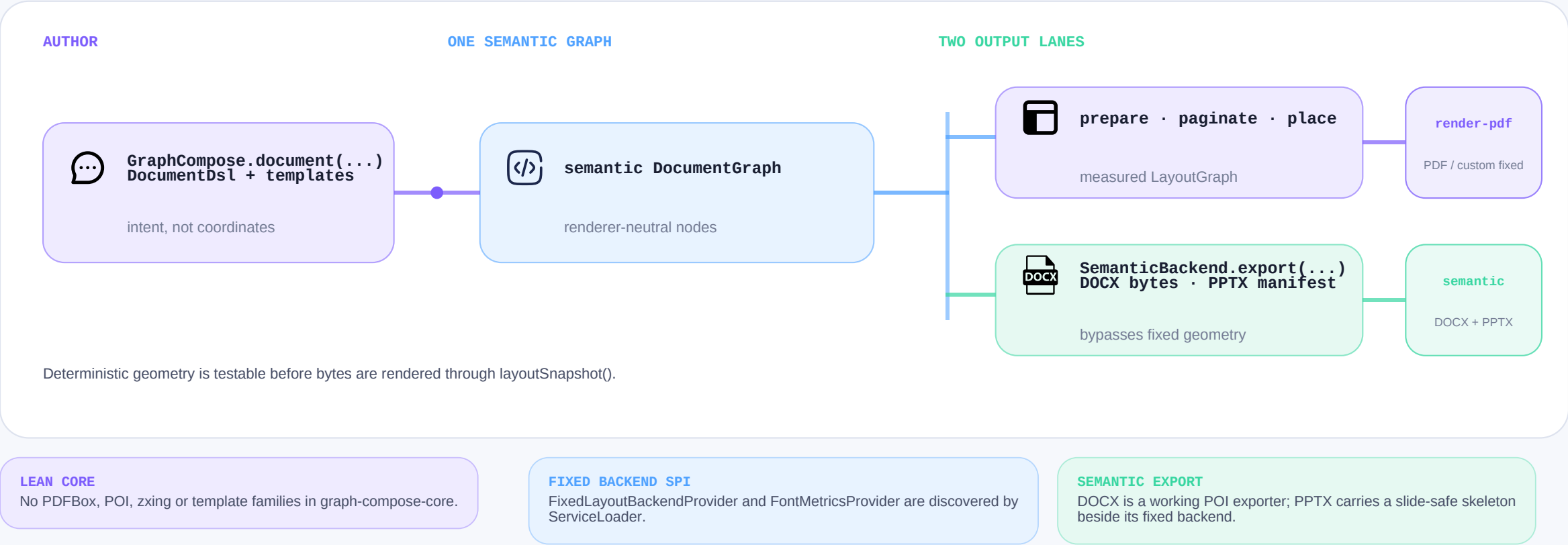
bring a renderer

● layoutSnapshot()

geometry regression

Semantic at the core. Format at the edge.

The canonical DSL produces one semantic DocumentGraph. Fixed-layout output resolves geometry; semantic exporters consume the graph directly.



The renderer boundary is explicit; Android or another backend is an extension opportunity, not a shipped 2.0 claim.

Choose only what you need.

Eight engine-train artifacts move in lockstep. Fonts and emoji keep independent version lines; heavy render and template dependencies are opt-in.

DROP-IN PDF DEFAULT

graph-compose

Core + PDF renderer. Canonical callers keeping this coordinate retain PDF out of the box.

LEAN / CUSTOM LEAN

graph-compose-core

Authoring, semantic model and layout services. Add a renderer module only when needed.

BATTERIES INCLUDED BUNDLE

graph-compose-bundle

Default PDF stack plus templates, bundled fonts and emoji. Office exporters stay opt-in.

THE LOCKSTEP TRAIN

One version, smaller dependency trees, explicit format boundaries.

● core

authoring + model

● render-pdf

fixed PDF

● render-docx

semantic DOCX

● render-pptx

fixed PPTX

● templates

opt-in presets

● testing

snapshot tools

● graph-compose

PDF wrapper

● bundle

PDF + templates + assets

Accepted migration trade-off: built-in templates are no longer carried by graph-compose; add graph-compose-templates or use the bundle.

2 / 4

Measured locally. Read with context.

Equivalent 1000-row report fixtures from the bundled 2026-08-04 single-machine snapshot. These values are directional, not a final 2.0 marketing benchmark.

40.0 ms

GraphCompose avg render
1000 rows

19.3 MiB

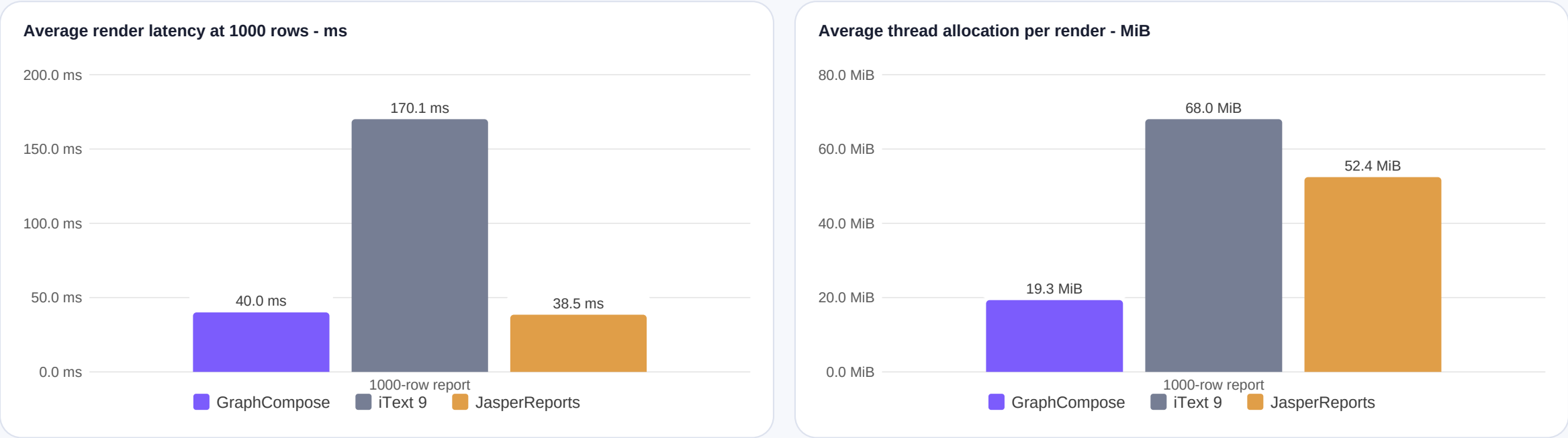
avg allocated / render
thread allocation

40 / 200 / 1000

report rows
scaling fixtures

2026-08-04

snapshot date
provenance incomplete

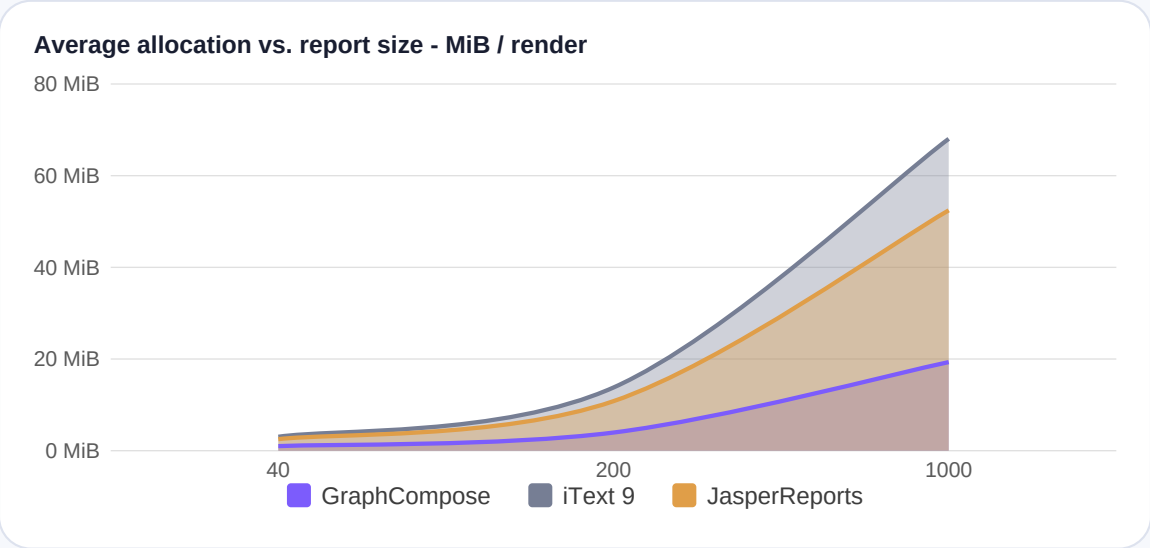
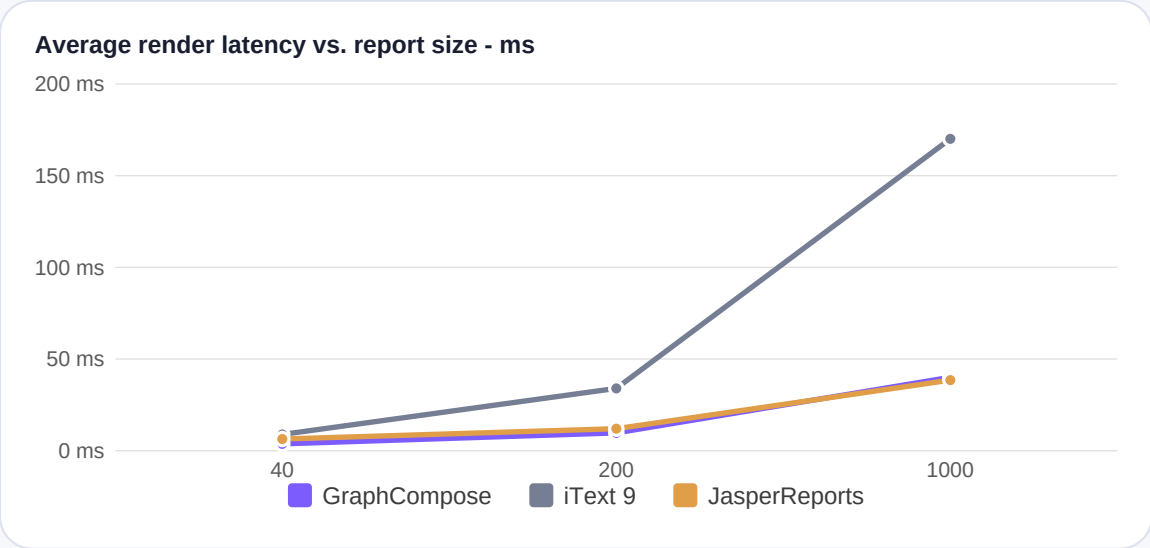


Scope: one developer machine, single run. The file records neither CPU, OS and JDK nor the commit measured, and its iteration counts cover only the small-invoice tier. Absolute milliseconds therefore travel badly between machines - read the ratios, not the timings.

Reproduce: scripts/run-benchmarks.ps1 - Current docs: docs/operations/benchmarks.md

Watch the curve, not one headline number.

The same local reference run across 40, 200 and 1000 rows. Allocation values are independent per-render measurements; they are not shares of a common heap.



4.25x
iText 9 took this many times longer in the 1000-row fixture

3.52x
iText 9 allocated this many times more in the same fixture

-4.0%
latency gap vs JasperReports - inside the documented 5-10% local noise band

What the 2.x line can claim today: **modular packaging, renderer isolation and deterministic layout snapshots**. The figures above are one machine's reference run, refreshed with the data file.

GraphCompose renders this deck itself: gradients, canvas layers, SVG, clipping and native charts.